

## Лабораторная работа № 8

**Тема:** Управление процессами, системными службами и мониторинг многозадачности в Linux и Windows Server

### Цель работы:

1. Научиться идентифицировать и анализировать ресурсы процессов (PID, CPU, RAM) в Linux и Windows Server.
2. Освоить управление жизненным циклом процессов: запуск в фоновом режиме, изменение приоритетов, отправка сигналов завершения.
3. Научиться диагностировать состояние системных служб и выявлять аномалии в многопоточных приложениях.

**Используемое ПО:** VM Linux (Debian/Ubuntu), VM Windows Server

### ТЕОРЕТИЧЕСКИЙ ЧЕК-ЛИСТ

Перед выполнением работы важно знать:

- **Вытесняющая многозадачность:** Планировщик ОС сам решает, когда переключать задачи и распределять кванты процессорного времени.
- **Процесс vs Поток:** Процесс — это тяжелая структура со своим изолированным адресным пространством и PID; потоки — легкие единицы выполнения внутри процесса, разделяющие его ресурсы.

### ЧАСТЬ 1. РАБОТА В ОС LINUX (Управление жизненным циклом и сигналами)

#### Этап 1: Снимок системы и интерактивный мониторинг

1. Выведите полный статический снимок запущенных в системе процессов с помощью команды:

```
ps aux
```

Найдите процесс вашего текущего командного интерпретатора bash (можно использовать `ps aux | grep bash`). Зафиксируйте его PID и имя пользователя, от чьего имени он запущен.

2. Запустите интерактивный монитор htop (или top, если htop не установлен).
3. Нажмите клавишу **F6** (в htop) для сортировки и выберите поле %MEM. Запишите имя процесса, который на данный момент является главным потребителем оперативной памяти. Нажмите q для выхода.

## Этап 2: Управление фоновыми процессами и приоритетами (Nice)

1. Запустите бесконечную команду фильтрации, которая будет имитировать долгую фоновую задачу, и сразу отправьте её в фон с помощью символа &:

```
ping 127.0.0.1 > /dev/null &
```

Система выведет номер задачи в квадратных скобках и PID процесса. Запишите этот PID.

2. Проверьте наличие фоновой задачи командой jobs.
3. Измените приоритет (nice) запущенного процесса, увеличив его дружелюбность к системе (снизив приоритет) до 15:

```
renice -n 15 -p <PID_вашего_ping>
```

(Убедитесь через `ps axo pid,ni,cmd | grep ping`, что значение Nice изменилось).

## Этап 3: Отправка сигналов (Жизненный цикл процесса)

1. Отправьте процессу сигнал остановки SIGSTOP (19), который переведет процесс в состояние Stopped (заморожен):

```
kill -19 <PID_вашего_ping>
```

2. Проверьте статус процесса через `ps aux | grep ping`. В колонке STAT вы увидите букву T (Stopped).
3. Завершите процесс принудительно, отправив самый жесткий сигнал ядра SIGKILL (9):

```
kill -9 <PID_вашего_ping>
```

### Этап 3: Аудит и инвентаризация системных служб в Linux

Службы (демоны) — это фоновые процессы, управляемые системой инициализации systemd. Администратор должен уметь получать их полный список и фильтровать по состоянию.

1. Выведите на экран полный список всех запущенных и активных системных служб (юнитов типа `.service`) в текущий момент:

```
systemctl list-units --type=service --state=running
```

2. Теперь выведите список всех установленных в системе служб, включая те, которые выключены или завершились с ошибкой:

```
systemctl list-unit-files --type=service
```

3. Найдите в этом списке службу планировщика задач `cron.service` (или `ssh.service`). Зафиксируйте в отчете её статус в колонке STATE (например, `enabled` — это значит, что служба автоматически стартует вместе с ядром ОС).

## ЧАСТЬ 2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ ДЛЯ WINDOWS SERVER (Анализ многопоточности и служб)

### Этап 1: Исследование потоков внутри процессов через Диспетчер задач

1. Откройте **Диспетчер задач** (Task Manager) → перейдите на вкладку «**Подробности**» (Details).
2. Нажмите правой кнопкой мыши по заголовкам таблиц (например, где написано *Имя*, *PID*) → выберите пункт «**Выбрать столбцы**» (Select columns).
3. Найдите и активируйте столбец «**Потоки**» (Threads). Нажмите ОК.
4. Отсортируйте процессы по количеству потоков. Найдите системный процесс **lsass.exe** (Local Security Authority Subsystem Service) или **services.exe**. Зафиксируйте в отчете его PID и количество выполняемых внутри него потоков.

### Этап 2: Мониторинг сетевых соединений процесса через Resource Monitor

Каждый запущенный процесс имеет право открывать файлы и сетевые порты.

1. Нажмите Win + R, введите resmon для запуска **Монитора ресурсов** (Resource Monitor).
2. Перейдите на вкладку **«Сеть»** (Network).
3. Разверните вкладку **«Сетевые подключения»** (Network Connections).
4. Найдите процесс веб-сервера (если установлен IIS, это будет w3wp.exe) или службы DNS/Active Directory. Если сервер «чистый», найдите процесс svchost.exe.
5. Запишите в отчет: имя процесса, его PID, локальный порт, который он слушает (Local Port), и адрес, с которым он связан.

### **Этап 3: Инвентаризация и аудит системных служб через GUI (Графический интерфейс)**

В Windows Server управление фоновыми процессами (службами) разделено между классической оснасткой «Службы» и Диспетчером задач. Администратор должен уметь быстро находить взаимосвязи между запущенной службой и реальным процессом в памяти.

#### **1. Поиск и анализ конкретной службы:**

- Нажмите комбинацию клавиш Win + R, введите **services.msc** и нажмите Enter. Откроется графическая оснастка **«Службы»** (Services).
- Найдите в алфавитном списке службу **«Диспетчер печати»** (Print Spooler).
- Нажмите по ней правой кнопкой мыши → **Свойства** (Properties).
- Зафиксируйте в отчете: *Имя службы (Service name)*, *Имя исполняемого файла (Path to executable)* и текущий *Тип запуска (Startup type)*.

#### **2. Связывание службы с процессом в памяти (PID):**

- Перейдите на вкладку **«Состояние»** в этом же окне свойств и убедитесь, что служба запущена (Running).

- Откройте **Диспетчер задач** (Ctrl + Shift + Esc) → перейдите на вкладку **«Службы»** (Services).
- Найдите в списке службу по её короткому имени Spooler.
- Посмотрите в столбец **PID** (Идентификатор процесса). Запишите это число.
- Нажмите по службе правой кнопкой мыши и выберите пункт **«Перейти к процессам»** (Go to details). Диспетчер задач автоматически перекинёт вас на вкладку «Подробности» и подсветит исполняемый .exe файл, внутри которого крутится эта служба. Запишите имя этого файла.

### 3. Поиск скрытых проблем (Остановленные автоматические службы):

- Вернитесь в окно services.msc.
- Нажмите на заголовок столбца **«Состояние»** (Status), чтобы отсортировать службы.
- Найдите глазами службы, у которых в колонке **«Тип запуска»** стоит **«Автоматически»** (Automatic), но при этом колонка **«Состояние»** (Status) пустая (то есть служба остановлена).
- *Выпишите в отчет имя любой одной такой службы, если она есть в системе.*

## ТРЕБОВАНИЯ К ОТЧЕТУ:

### 1. По разделу Linux:

- Скриншот вывода команды ps aux с выделенной строкой процесса bash и его PID.
- Скриншот выполнения команд renice и kill -9.
- Скриншот вывода команды systemctl list-units --type=service --state=running (первые 10–15 строк).

### 2. По разделу Windows Server:

- Скриншот вкладки «Подробности» Диспетчера задач с отображением добавленного столбца «Потоки».

- Таблица по результатам мониторинга сетевых соединений из Монитора ресурсов (Имя процесса, PID, Локальный порт).
- Скриншот окна Свойств службы «Диспетчер печати» (Print Spooler), где четко виден путь к исполняемому файлу на диске.
- Скриншот Диспетчера задач на вкладке «Подробности» (Details) с подсвеченным процессом, который вы нашли через контекстное меню службы.

### Шпаргалка-памятка:

- **В Linux:** Помните, что zombie-процесс — это не «живой» процесс, он не потребляет процессорное время (CPU) или оперативную память, а является лишь мертвой записью в таблице процессов, которую родитель забыл считать с помощью системного вызова `wait()`. Убить зомби командой `kill -9` невозможно, так как он уже мертв. Нужно перезапустить его родительский процесс.
- **В Windows Server:** Столбец «Потоки» в Диспетчере задач наглядно демонстрирует, почему один процесс (например, браузер или СУБД) может одновременно выполнять много задач (скачивать файл, отрисовывать страницу, слушать сеть) — под каждую подзадачу выделяется свой легковесный поток внутри общего адресного пространства процесса